

Arrange your messy dates & times : : CHEAT SHEET

Retain, represent, and reason about date/time imprecision — an implementation of ISO 8601-2:2019 (EDTF) for R.



Basics: annotate dates & times

The `mdate` class embeds EDTF annotations, so one value can express imprecision instead of forcing a false precise date:

?	uncertain	2008-01-18?
~	approximate	2012-09-12~
%	uncertain & approximate	
X	unspecified	2012-XX-18
..	range	2012-01..2012-03
{ } []	set (all) / one-of	
T	time + zone	...T14:30Z

1. Coerce objects to `mdate`

`as_messydate(x)` coerces `Date`, `POSIXct`, `POSIXlt`, character, or written text (dates *and* times) into `mdate`:

input	mdate
"2021-04"	2021-04
"2012-09-12~"	2012-09-12~
"2012-01..2012-03"	2012-01..2012-03
"first of Feb 2012"	2012-02-01
"2019-03-01 2:30pm"	2019-03-01T14:30

`make_messydate(year, month, day)` builds from columns: (2007, 2, 17) → 2007-02-17.

2. Expand & contract

`expand(md)` opens a messy date into *all* compatible dates (day granularity; use `by="hour"/"min"/"sec"` for times):

- "2021-04" → 30 dates 2021-04-01 ... -04-30
- "2019-02-01..2019-02-03" → 3 dates

`contract(md)` is the inverse — the most concise form: "2021-04-01..2021-04-30" → 2021-04.

3. Resolve to one date/time

Coerce back with `as.Date()`, `as.POSIXct()`, or `as.POSIXlt()` (the POSIX methods keep the time). Choose *how* to resolve with `FUN =`:

`min` `max` `mean` `median` `modal` `random` (whole vector), or vectorised `vmin` `vmax` `vmean` `vmedian` `vmodal` `vrandom` (element-wise).

`as.Date(as_messydate("2021-04"), vmin)` → 2021-04-01
..., `vmax`) → 2021-04-30
..., `vmean`) → 2021-04-16



⌚ Times (new in 1.0.0)

Times of day use the ISO `T` separator, and take the same annotations as dates:

- **Parse:** "2019-03-01T14:30:00Z", am/pm, +02:00 offsets, fractional seconds.
- **Extract:** `hour()` `minute()` `second()` `tz()`.
- **Annotate a part:** `as_approximate(x, "hour")` → ...T~14:30.
- **Precision below a day:** `precision("...T14:30")=1440` (24 hr, 1440 min, 86400 sec).
- **Shift & step:** `md + "2 hours"; seq(a, b, by="hour")`.

Annotate uncertainty

`on_or_before(x)` → ..x
`on_or_after(x)` → x..
`as_approximate(x)` → x~
`as_uncertain(x)` → x?

Add `component=` to mark one part: "year" "month" "day" "hour" "minute" "second", or groups "ym" "md" "time".

Combine two `mdates`

- `x %intersect% y` — dates common to both
- `x %union% y` — dates in either
- `x + y` — multiset (union of ranges/sets)

Logical tests

`is_messydate(x)` `is_precise(x)` `is_uncertain(x)`
`is_approximate(x)` `is_bce(x)`
`is_intersecting(x,y)` `is_subset(x,y)` `is_similar(x,y)`

Proportional comparison

Proportion of `x`'s possible dates meeting a test against `y`:

- `x %l% y` before min `x %le% y ≤`
- `x %g% y` after max `x %ge% y ≥`
- `x %><% y` within `x %>=<% y` within, incl.

Components & durations

Extract `year()` `month()` `day()` `hour()` `minute()` `second()` `tz()`, or measure `precision()`.

`messyduration(range)` returns a duration that accounts for the uncertainty at each end.